```
import torch
import torch.nn as nn
import numpy as np
import matplotlib.pyplot as plt

# Tetapkan seed untuk reproduktibilitas
torch.manual_seed(42)
np.random.seed(42)

# ========================================
# BAGIAN 1: MEMBUAT DATASET TIME SERIES (SINE WAVE)
# ========================================

# 1. Membuat data gelombang sinus (1000 titik data)
t = np.linspace(0, 100, 1000)
y = np.sin(t)

# 2. Fungsi Sliding Window
def create_sequences(data, seq_length):
    xs = []
    ys = []
    # Geser jendela sepanjang data
    for i in range(len(data) - seq_length):
        # Ambil sejumlah 'seq_length' data masa lalu
        x_window = data[i:(i + seq_length)]
        # Target adalah 1 titik data tepat setelah jendela tersebut
        y_target = data[i + seq_length]

        xs.append(x_window)
        ys.append(y_target)

    return np.array(xs), np.array(ys)

SEQ_LENGTH = 10 # Model akan melihat 10 titik terakhir untuk menebak titik ke-11

X, y_target = create_sequences(y, SEQ_LENGTH)

# 3. Konversi numpy array ke PyTorch Tensor
# PyTorch RNN mengharapkan dimensi: (Batch_Size, Sequence_Length, Input_Features)
# Karena data kita 1 dimensi (hanya nilai Y), Input_Features = 1
X_tensor = torch.tensor(X, dtype=torch.float32).unsqueeze(-1)
y_tensor = torch.tensor(y_target, dtype=torch.float32).unsqueeze(-1)

# Bagi menjadi Data Latih (80%) dan Data Uji (20%)
train_size = int(len(X_tensor) * 0.8)
X_train, X_test = X_tensor[:train_size], X_tensor[train_size:]
y_train, y_test = y_tensor[:train_size], y_tensor[train_size:]

print(f"Bentuk X_train: {X_train.shape} -> (Batch, Seq_Len, Features)")
print(f"Bentuk y_train: {y_train.shape}")
```

Show code

```
# ========================================
# BAGIAN 2: MEMBANGUN MODEL RNN UNTUK REGRESI
# ========================================

class RNNTimeSeries(nn.Module):
    def __init__(self, input_size=1, hidden_size=32, output_size=1):
        super(RNNTimeSeries, self).__init__()

        self.hidden_size = hidden_size

        # Layer RNN Utama
        # batch_first=True berarti dimensi pertama adalah ukuran batch
        self.rnn = nn.RNN(input_size, hidden_size, batch_first=True)

        # Layer Fully Connected untuk output (Regresi)
        self.fc = nn.Linear(hidden_size, output_size)

    def forward(self, x):
        # x shape: (batch_size, seq_length, input_size)

        # output shape: (batch_size, seq_length, hidden_size)
        # hidden shape: (1, batch_size, hidden_size)
        out, hidden = self.rnn(x)

        # Karena kita hanya ingin memprediksi nilai MASA DEPAN,
        # kita HANYA mengambil output dari langkah waktu (time-step) TERAKHIR.
        # out[:, -1, :] mengambil semua batch, time-step terakhir (-1), semua fitur hidden.
        out_last_step = out[:, -1, :]

        # Lewatkan ke layer linear untuk mendapatkan 1 nilai prediksi
        hasil = self.fc(out_last_step)
        return hasil

# Inisialisasi Model
```

```
model = RNNTimeSeries(input_size=1, hidden_size=32, output_size=1)
print(model)
```

[ ]

```
# ==========================================
# BAGIAN 3: TRAINING LOOP (MSE LOSS)
# ==========================================

# Loss untuk Regresi (menebak nilai riil)
criterion = nn.MSELoss()
# Optimizer Adam
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)

epochs = 150

print("Memulai Training...")
for epoch in range(epochs):
    model.train()

    # Forward pass
    predictions = model(X_train)

    # Hitung error / loss
    loss = criterion(predictions, y_train)

    # Backward pass & Optimize
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    if (epoch+1) % 20 == 0:
        print(f'Epoch [{epoch+1}/{epochs}], Loss: {loss.item():.6f}')
```

[ ]

```
# ==========================================
# BAGIAN 4: PREDIKSI DAN VISUALISASI
# ==========================================

model.eval() # Matikan dropout/batchnorm jika ada
with torch.no_grad():
    # Prediksi menggunakan data latih dan data uji
    train_predictions = model(X_train).numpy()
    test_predictions = model(X_test).numpy()

# Menggabungkan prediksi untuk diplot
# (Perlu menambahkan NaN agar posisi index grafik sesuai dengan urutan waktu asli)
plot_train = np.empty_like(y_target)
plot_train[:] = np.nan
plot_train[:train_size] = train_predictions.flatten()

plot_test = np.empty_like(y_target)
plot_test[:] = np.nan
plot_test[train_size:] = test_predictions.flatten()

# Visualisasi dengan Matplotlib
plt.figure(figsize=(14, 5))
plt.title("RNN Time Series Forecasting (Sine Wave)")
# Plot target asli yang harus ditebak
plt.plot(y_target, label='Data Aktual (True Value)', color='blue', alpha=0.5)
# Plot hasil prediksi dari Data Latih
plt.plot(plot_train, label='Prediksi Training', color='green')
# Plot hasil prediksi dari Data Uji (Masa Depan / Unseen Data)
plt.plot(plot_test, label='Prediksi Testing (Masa Depan)', color='red')

plt.axvline(x=train_size, c='black', linestyle='--', label='Batas Train/Test')
plt.legend()
plt.show()
```

[ ]

```
Start coding or generate with AI.
```